

Instructions. This homework involves both a write-up and an SQL coding component and is worth 15 points. Submit the following three files to Canvas by 11:59 PM on the due date:

- `hw3_1.sql`: your script to create and populate the tables in Part 1.
- `hw3_2.sql`: your script containing the ten queries in Part 2, in question order.
- `hw3_3.pdf`: your typed write-up described in Part 3. Organize it by question number and include your first and last name, the course (CPSC-321, Fall 2026), and the assignment (HW-3) at the top.

SQL Scripts. Begin each SQL script file with a header comment in the following format, replacing the name with your own:

```
/*=====
* NAME:      Your first and last name
* ASSIGN:    HW-3, Part 1
* COURSE:    CPSC 321, Fall 2026
* DESC:      Create and populate the "factbook" database tables.
*=====*/
```

For `hw3_2.sql`, use the same header format, but change `ASSIGN` to `HW-3, Part 2` and `DESC` to a brief description of that file's queries. Format each file based on one of the SQL style guides linked from the course webpage, and apply it consistently throughout both scripts. Include a comment describing each table's purpose. Include the query comments required in Part 2. Missing headers, poor formatting, or failure to follow the submission instructions may result in a reduction in points.

PART 1. Implement the following schema using `CREATE TABLE` statements in your PostgreSQL database. The schema is loosely based on the CIA World Factbook.¹ Your statements must go into `hw3_1.sql`. You must pick suitable attribute data types and include primary keys, foreign keys, and constraints as specified below.

`country(country_code, country_name, gdp, inflation)`

- A country has a two-letter country code (e.g., "US"), a full name (e.g., "United States of America"), a gross domestic product per capita (e.g., 46,900 dollars per person), and an inflation rate (e.g., 3.8 percent), where `country_code` is the primary key. Assume that two countries with different country codes can have the same country name. All attributes are required.

`province(province_name, country_code, area)`

- A province (which is referred to as a "state" in the US) consists of a name (e.g., "Washington"), the code of the country where the province is located, and the total province area in km². The `province_name` and `country_code` together form the primary key, with `country_code` a foreign key to the `country` table. Assume it is possible for two countries to have a province with the same name (e.g., Montana exists in both the US and Bulgaria). All attributes are required.

¹See: <https://worldfactbookarchive.org/>, previously: <https://www.cia.gov/the-world-factbook/>

`city(city_name, province_name, country_code, population)`

- A city is identified by its name, province, and country, and has a total population. The `province_name` and `country_code` together define a foreign key to the `province` table. Assume it is possible for two provinces to have a city with the same name (e.g., Portland is a city in both Oregon and Maine). All attributes are required.

`border(country_code_1, country_code_2, border_length)`

- A border defines a connection between two countries, with a corresponding border length in km. Both `country_code_1` and `country_code_2` are separately foreign keys to the `country` table. The two country codes must differ. Include a constraint to enforce this requirement. Assume there is only one row for each border between two countries. For example, a border between countries *A* and *B* may be stored as either (*A*, *B*) or (*B*, *A*), but not both. Either ordering is permitted. All attributes are required.

Populate your tables using `INSERT INTO` statements with enough data to test your queries. At a minimum, you must include at least four different countries, two different provinces per country, and three different cities per province. You must also include at least three borders. Include your insert statements in your `hw3.1.sql` file. *Your data may be entirely made up.* Country, province, and city names, country codes, and all numeric values do not need to match actual examples. Choose data that satisfies the schema constraints and lets you test the queries below. If you use real places, you do not need to include all of their provinces or cities.

PART 2. Each question below asks you to write an SQL query against your world factbook tables from Part 1 above. For some queries you may need to add additional rows to your tables. In general, you must add enough rows to ensure your queries are working correctly (which may take some thought). Each query must return at least two rows. Your data must also include a nonmatching example for each query, as described in Part 3.

Place your answers to Queries 1–10 in `hw3.2.sql`. As part of grading, your scripts will be executed in PostgreSQL, i.e., `hw3.1.sql` will be run to create your tables and load your submitted data, and then `hw3.2.sql` to execute your queries. Both scripts must run without errors. Your `hw3.1.sql` script must drop existing tables before recreating and populating them so that it can be run more than once without errors. Include all data in `hw3.1.sql` needed by your queries. Test the final versions of both scripts before submitting them. The query results must match those reported in your write-up (row order may differ where sorting is not required). For each query in `hw3.2.sql`, you must provide a *comment* giving:

- the question number (Query 1–10);
- a description of the purpose of the query (e.g., based on the question); and
- any other relevant comments or any assumptions you made.

Where a query below asks you to choose a threshold or a range, pick values that make sense for your data and state them in the query's comment. Use the same values when rewriting a query with `JOIN` syntax. For full credit, when writing your queries you must:

- (i) *use as few joins as possible (i.e., do not include unnecessary tables in your FROM clauses);*

- (ii) *only use* `DISTINCT` *and* `ORDER BY` *when needed*; and
- (iii) *only use the SQL constructs and syntax listed below.*

For this assignment, you may use the following constructs. These are permitted options, and you do not need to use all of them:

- `SELECT`, `FROM`, and `WHERE`, including arithmetic expressions, comparisons, logical operators, and table and column aliases.
- Comma joins, `JOIN...ON`, `JOIN...USING`, and `NATURAL JOIN`.
- `DISTINCT` and `ORDER BY` where needed.

Follow each question's requirements concerning join syntax. Do not use subqueries, aggregate functions, `GROUP BY`, `HAVING`, set operations (e.g., `UNION`), outer joins, `CROSS JOIN`, or `WITH` clauses.

Finally, be sure you give each answer below in the order given, and in general, try to make grading your work easy for the grader. For questions below that require the use of a "comma join" (i.e., without `JOIN` syntax), you must include the appropriate join conditions in the `WHERE` clause.

1. Write an SQL query using a comma join to find provinces with an area greater than a threshold you choose that are in countries with a GDP per capita greater than a threshold you choose. Return the country code, country name, GDP per capita, province name, and province area. Sort the results from highest to lowest GDP per capita, then alphabetically by country code, and then from largest to smallest province area.
2. Rewrite your query from (1) using appropriate `JOIN` syntax instead of a comma join. Preserve the output columns, conditions, and sorting.
3. Write an SQL query using comma joins to find provinces whose area is less than a threshold you choose and that have at least one city with a population within a range you choose (inclusive). Return the country code, province name, and province area, with only one row per matching province.
4. Rewrite your query from (3) using `JOIN` syntax. Preserve the output columns and conditions, and return only one row per matching province.
5. Write an SQL query using comma joins to find provinces that have at least two different cities with populations greater than a threshold you choose and that are in countries with inflation below a threshold you choose. Return the country code, country name, inflation, province name, and province area. Return only one row per matching province.
6. Rewrite your query from (5) using `JOIN` syntax for all joins. Preserve the output columns and conditions, and return only one row per matching province.
7. Write an SQL query using `JOIN` syntax to find pairs of cities with the same name that are in different countries and have different populations. For each city, return its name, province name, country code, and population (eight attributes total). Return each pair only once, with the less populous city appearing first in the output row. Give each output column a different name that identifies which city it describes (for example, `population_1` and `population_2`).
8. Write an SQL query using comma joins to find countries that have a higher GDP per

capita and a lower inflation rate than at least one country they border, where the shared border is longer than a threshold you choose. Return the country code and country name, with only one row per matching country. A border may be stored with either country in `country_code_1`. Your query must find qualifying countries whether they appear in `country_code_1` or `country_code_2`.

9. Rewrite your query from (8) using JOIN syntax for all joins. Preserve the output columns and conditions, and return only one row per matching country.
10. Develop your own query that answers a different question from (1)–(9). It must use JOIN syntax to combine at least three different tables, include at least two non-join conditions, and return a subset of the stored attributes. The result must contain only unique rows and must be sorted in an order appropriate to your question. In the query’s comment, describe the question and why you chose the sorting order. Use only constructs permitted for this assignment.

PART 3. For each query in Part 2, your `hw3_3.pdf` file must include the following:

1. **Query number**, along with any chosen thresholds or ranges.
2. **Actual output** copied from PostgreSQL. Use a monospaced font and preserve the column headings and row alignment.
3. **Matching example:** identify one row in the output, show the input rows that produce it, and briefly explain why it qualifies. Show the actual input values in small tables or copy them from your SQL script. A few sentences explaining the example are sufficient.
4. **Nonmatching example:** identify an example based on data actually stored in your tables that should not appear in the query result, and confirm that it is absent from the output.

For example, for a query that requires at least two cities meeting a condition, a useful matching example is a province with two qualifying cities. A useful nonmatching example is a province with only one qualifying city.

You may need to add data to your `hw3_1.sql` script to fully test each query. Run all queries against the final data created by your submitted `hw3_1.sql` script. For Queries 2, 4, 6, and 9, reuse the examples from the corresponding original query, but include the output from each version and confirm that both versions produce the same results. For queries that do not require sorting, row order may differ.